

# CS6005 Portfolio — Sprint 3 (Innovative Feature) Update Plan

This document captures the changes needed to bring the portfolio in line with the *actual* innovative feature that has been implemented in the codebase (the Pause & Reflect notification → re-decision loop), and to remove the stale "Insights / Review Spending" content that is currently sitting in the Sprint 3 test cases.

## 1. Diagnosis of the current portfolio

Read of CS6005\_Portfolio\_ZACHARY\_BECK.pdf:

| Slide  | Section                                         | Status                                                                                                                                                                                                                                                                                              |
|--------|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 16     | \$2.3.5 Sprint 3 shall-statements (S3.1 – S3.6) | <b>Already describe the innovative feature correctly.</b> S3.1 reflect sheet, S3.2 wait reminder with deep-link re-decision, S3.3 wish-list pot / skip, S3.4 single-record data integrity (NFR), S3.5 on-device LLM (NFR), S3.6 <100 ms pre-fill (NFR).                                             |
| 18     | \$2.3.6 Sprint 3 test cases (T3.1 – T3.8)       | <b>Wrong.</b> Every test on this slide is for <i>Review Spending / Insights</i> (chart loading, filters, drill-down, VoiceOver on insights). None of them tests S3.1 – S3.6. This is the "insights stuff" to rip out.                                                                               |
| 7 – 10 | \$2.1 Use Case Diagrams                         | Main diagram + sub-diagrams already cover Pause & Reflect, Wait on it, Re-decision notification. <b>Two small fixes:</b> (a) the <b>Skip</b> action is missing from the diagram (it's mentioned in the use-case table but not drawn), (b) re-decision notification arrow can be drawn more clearly. |
| 17     | S3.4 description                                | Self-flagged: // <b>Need to look into this one more.</b> Needs a real description (see §3 below).                                                                                                                                                                                                   |

So the work is narrow and well-defined: replace T3.1 – T3.8, redraw the Pause-before-purchase use case diagram with **Skip**, and write S3.4 properly.

## 2. Updated use case diagrams (PlantUML)

Two diagrams change. Top-level diagram stays identical to what's currently on slide 7 (it already only shows the three top-level use cases). The **Pause-before-purchase** sub-diagram needs **Skip** added and the wait durations matched to the code (**InterventionWaitDuration**).

### 2.1 Top-level use case diagram (unchanged — provided here for completeness)

```
@startuml UseCase_TopLevel
left to right direction
skinparam packageStyle rectangle

actor User

rectangle Bujet {
    usecase "Open Bujet" as Open
    usecase "Manual transaction entry" as Manual
    usecase "Transaction import" as Import
    usecase "Pause & reflect before purchase" as PauseReflect

    User --> Open
    Open ..> Manual : <<extend>>
    Open ..> Import : <<extend>>
    Open ..> PauseReflect : <<extend>>
}
@enduml
```

### 2.2 Pause-before-purchase sub-diagram (updated — replaces current slide 10)

```
@startuml UseCase_PauseBeforePurchase
left to right direction
skinparam packageStyle rectangle

actor User
```

```

rectangle "Pause & reflect before purchase" {
    usecase "Enter amount, description\n& category" as Enter
    usecase "View reflection\n(budget impact + LLM narrative)" as Reflect

    usecase "Buy now" as BuyNow
    usecase "Add to wish-list pot" as Pot
    usecase "Wait on it\n(24h / 7 days / end of month)" as Wait
    usecase "Skip\n(re-decision only)" as Skip

    usecase "Schedule re-decision\nnotification" as Schedule
    usecase "Tap notification →\nreopen pre-filled sheet" as Redecision

    User --> Enter
    Enter ..> Reflect : <<include>>

    Reflect <.. BuyNow : <<extend>>
    Reflect <.. Pot : <<extend>>
    Reflect <.. Wait : <<extend>>

    Wait ..> Schedule : <<include>>
    Schedule ..> Redecision : <<include>>
    Redecision ..> Reflect : <<include>>

    Reflect <.. Skip : <<extend>>
    note bottom of Skip
        Visible only when the sheet was
        opened from a wait-reminder tap.
        Resolves the pending wait as
        "skipped" – counts as money saved,
        no pot, no transaction.
    end note
}
@enduml

```

### 2.3 Optional new sequence diagram for §3.3 (PlantUML)

You currently have message-sequence diagrams for Connect-bank, Manual-add, and Review-spending. The Review-spending one is dead weight if you're removing the insights focus from Sprint 3. **Suggest replacing it with the diagram below**, which shows the innovative loop end-to-end:

```

@startuml Sequence_PauseReflect_WaitRedecision
actor User
participant "PauseReflect\nInputView" as Input
participant "PreSpend\nInterventionFlow" as Flow
participant "PreSpend\nInterventionUseCase" as UseCase
participant "WaitReminder\nScheduler" as Sched
participant "UNUserNotification\nCenter" as UN
participant "NotificationCenter\nDelegate" as Del
participant "WaitReminder\nRouter" as Router
participant "InterventionLog\nRepository" as LogRepo

== First decision ==
User -> Input : enter amount, description, category
Input -> Flow : submitInput()
Flow -> UseCase : evaluate(proposal)
UseCase --> Flow : InterventionDecision
Flow --> Input : show decision view

User -> Flow : chooseWait(24h)
Flow -> UseCase : recordDecision(.wait)
UseCase -> LogRepo : append(pending wait log)
Flow -> Sched : schedule(proposal, duration)
Sched -> UN : add(UNNotificationRequest)
Flow --> Input : step = .waitConfirmed

== Notification fires ==

```

```
UN -> Del : didReceive(response)
Del -> Router : present(prefilledProposal)
Router -> Flow : new flow with prefilledProposal
Flow -> Flow : start() → runEvaluation()
Flow --> Input : step = .decision (auto)

== Re-decision: Skip ==
User -> Flow : chooseSkip()
Flow -> LogRepo : resolveWait(id, outcome: .skipped)
note right of LogRepo
  Single canonical record:
  the pending wait is *resolved*,
  not duplicated. The £ amount
  counts once toward "saved by
  pausing", regardless of how many
  times the user snoozed the item.
end note
Flow --> Input : dismiss
@enduml
```

3. Replacement text for S3.4 description (slide 17)

Current text: */// Need to look into this one more (not sure what it means)."*

Replace with:

**S3.4: The app shall save the "Money saved" amount as a single canonical record, enhancing data integrity (NFR).**

Description: Each "Wait on it" decision creates exactly one pending **InterventionLog** record, keyed by the original proposal ID. When the notification fires and the user re-decides, the existing record is *resolved* (**.purchased**, **.skipped**) rather than a new one being written. This guarantees that snoozing the same £35 jumper five times still contributes £35 to the "saved by pausing" total — not £175. The metric stays honest under repeated wait/snooze cycles.

This NFR is implemented in **PreSpendInterventionFlow.resolvePendingWait** (**Features/PreSpendIntervention/PreSpendInterventionFlow.swift:291**) and gated by **isRedecision** on **chooseBuyNow**, **chooseAddToPot**, **chooseSkip**, and **chooseWait**.

4. Replacement Sprint 3 test cases (replaces T3.1 – T3.8 on slide 18)

Mapped 1:1 against S3.1 – S3.6. Each test is grounded in code that exists.

| Test ID | Feature ID & brief description                                                | Input operations (sequence + data)                                                                                                                                   | Expected output                                                                                                                                                                                                                                                       |
|---------|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T3.1    | S3.1 Pause & Reflect sheet opens and accepts a valid proposal.                | 1) Launch app. 2) Tap "I'm about to spend...". 3) Enter Amount = <b>35</b> , Description = <b>Wool jumper</b> , Category = <b>Shopping</b> . 4) Tap <b>Reflect</b> . | Decision view appears showing: budget impact for Shopping, LLM-generated narrative paragraph, four-button choice ( <b>Buy now</b> , <b>Wait on it</b> , <b>Add to Pot</b> , no <b>Skip</b> on first-decision). No <b>Skip</b> button visible (it's re-decision only). |
| T3.2    | S3.1 Validation — Reflect is disabled for zero / empty input (negative test). | 1) Tap "I'm about to spend...". 2) Leave amount blank (or <b>0</b> ) and/or description blank. 3) Observe the Reflect button state.                                  | <b>Reflect</b> button is disabled. If user forces submission, inline error appears ("Enter a valid amount." / "Add a short description.") and no decision view is shown.                                                                                              |
| T3.3    | S3.1 No-budget nudge fires when the chosen                                    | 1) Ensure no budget is set for <b>Shopping</b> . 2) Open the sheet, enter £35 / "Wool jumper" / Shopping. 3) Tap Reflect.                                            | "Set a budget?" alert appears with two options: <b>Set a budget</b> (presents <b>BudgetsSheet</b> on top of the flow) and <b>Continue without budget</b> (proceeds to                                                                                                 |

| Test ID | Feature ID & brief description                                                                                                                                          | Input operations (sequence + data)                                                                                                                                                                                                                   | Expected output                                                                                                                                                                                                                                                                                                                                                 |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         | category has no limit set.                                                                                                                                              |                                                                                                                                                                                                                                                      | evaluation). Input is preserved across the alert.                                                                                                                                                                                                                                                                                                               |
| T3.4    | <b>S3.2</b> "Wait on it" schedules a local notification with the proposal encoded in <code>userInfo</code> .                                                            | 1) Complete T3.1 to land on the decision view. 2) Tap <b>Wait on it</b> → choose <b>24 hours</b> . 3) Inspect pending notifications ( <code>UNUserNotificationCenter.getPendingNotificationRequests</code> ) or wait for fire (DEBUG fires in ~5 s). | One <code>UNNotificationRequest</code> exists with identifier <code>wait-&lt;proposalID&gt;</code> . Its <code>userInfo</code> contains <code>kind = wait_reminder</code> , plus <code>proposalID</code> , <code>amount</code> , <code>itemDescription</code> , <code>category</code> , <code>currencyCode</code> . Title reads "Still want Wool jumper?".      |
| T3.5    | <b>S3.2</b> Tapping the wait-reminder banner deep-links into a pre-filled Pause & Reflect sheet sitting on the decision view.                                           | 1) Complete T3.4 and background the app. 2) Wait for the banner to fire (~5 s in DEBUG). 3) Tap the banner.                                                                                                                                          | App foregrounds; reflect sheet opens automatically; amount field shows <b>35</b> , description shows <b>Wool jumper</b> , category shows <b>Shopping</b> ; the user lands directly on the decision view (input step is skipped); a <b>Skip</b> button is now visible alongside Buy now / Wait / Add to Pot.                                                     |
| T3.6    | <b>S3.3</b> "Add to Pot" on the re-decision flow resolves the pending wait as <code>.skipped</code> and creates a wishlist pot (does <b>not</b> write a duplicate log). | 1) From the deep-linked re-decision sheet (T3.5), tap <b>Add to wish-list pot</b> .                                                                                                                                                                  | A new <code>Goal</code> is created with <code>targetAmount = 35</code> , <code>linkedItem.createdFromIntervention = &lt;proposalID&gt;</code> . The original pending <code>InterventionLog</code> for that proposal moves to <code>outcome = .skipped</code> . No second log is appended. "Saved by pausing this month" increases by £35 (one record, not two). |
| T3.7    | <b>S3.3</b> "Skip" on the re-decision flow resolves the pending wait as <code>.skipped</code> with no pot and no transaction.                                           | 1) From the deep-linked re-decision sheet (T3.5), tap <b>Skip</b> .                                                                                                                                                                                  | Sheet dismisses. The pending <code>InterventionLog</code> for <code>&lt;proposalID&gt;</code> is now <code>.skipped</code> . No <code>Goal</code> exists for the item; no <code>Transaction</code> is written. "Saved by pausing this month" increases by £35.                                                                                                  |
| T3.8    | <b>S3.4</b> Repeated snoozing does <b>not</b> double-count toward "saved by pausing" (single-record data-integrity NFR).                                                | 1) Complete T3.4 → T3.5 (1st re-decision). 2) On the re-decision sheet, tap <b>Wait on it</b> → 24 h. 3) When the banner fires again, tap it. 4) Tap <b>Skip</b> .                                                                                   | Across the whole sequence, exactly <b>one</b> <code>InterventionLog</code> exists for the proposal. Its final state is <code>.skipped</code> . "Saved by pausing this month" reflects £35, not £70.                                                                                                                                                             |
| T3.9    | <b>S3.5</b> On-device LLM narrative — no proposal data leaves the device.                                                                                               | 1) Put the simulator into Airplane mode (or block network at the OS level). 2) Run T3.1 end-to-end. 3) Inspect the network traffic logs and the decision-view narrative paragraph.                                                                   | A non-empty LLM narrative is rendered. Zero outbound network traffic during evaluation. If the on-device model fails to load (e.g., older simulator), the app falls back to hard-coded copy with the same numerical budget impact — sheet still renders, no crash.                                                                                              |

| Test ID | Feature ID & brief description                                                                                                 | Input operations (sequence + data)                                                                                                                                                                                                                                 | Expected output                                                                                                                                                                                                                               |
|---------|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T3.10   | <b>S3.6</b><br>Notification → pre-fill latency under 100 ms (Apple HIG responsiveness).                                        | 1) From a cold-start state, fire the wait banner (~5 s after Wait in DEBUG). 2) Tap the banner. 3) Use Instruments → Time Profiler / signposts around <code>WaitReminderRouter.present(_:)</code> and the <code>PreSpendInterventionFlow.init</code> that follows. | Time from <code>WaitReminderRouter</code> receiving the prefilled proposal to the decision view becoming visible is < 100 ms (excludes LLM evaluation, which runs after the sheet is on screen). No dropped frames during sheet presentation. |
| T3.11   | <b>S3.2</b> Cold-start sheet-collision: banner tap during another modal sheet correctly defers and presents the reflect sheet. | 1) Open the Budgets sheet (any non-reflect modal). 2) Trigger the wait banner. 3) Tap the banner while the other sheet is on screen.                                                                                                                               | The other sheet is dismissed (or the reflect sheet is presented on top per the implementation in <code>MainTabView</code> ), and the reflect sheet opens pre-filled with the proposal. No silent drop, no double-presentation crash.          |

**Note on table size:** the rubric only requires test cases that demonstrate coverage of the shall-statements. 11 tests across 6 statements is generous; you can drop T3.11 or T3.9 if space is tight.

## 5. Slide-by-slide change list (what to actually edit in the .pptx)

- Slide 10** ("Pause before purchase" use case diagram) — redraw using the PlantUML in §2.2. Add the `Skip` use case + note.
- Slide 17** (Sprint 3 shall-statements) — replace S3.4 description with the text in §3.
- Slide 18** (Sprint 3 test cases) — delete the entire table, replace with the 9-to-11 row table in §4.
- Slide 23** (Review Spending sequence diagram) — *optional but recommended*: swap for the sequence diagram in §2.3 so your sequence diagrams cover all three Sprint features (Sprint 1 connect, Sprint 2 manual add, Sprint 3 innovative loop). If you keep the Review Spending sequence diagram, the portfolio narrative becomes "Sprint 3 is insights" which contradicts S3.1 – S3.6.
- Slide 12** (Use case table for Pause & review) — already mentions the Skip alternative, but the wording `4iii(alt)` is slightly off. Suggest cleaning up to: *"Alternatives: 4iv: On a wait-reminder re-decision the sheet shows an additional `Skip` button which resolves the pending wait as 'skipped' without creating a pot or transaction."*

Sections 1, 2, 3 (project ideation, requirements, architecture intro) and the Sprint 1 / Sprint 2 test cases are untouched.

## 6. Things I noticed but did **not** change (flagging for your call)

- Class diagram (slide 21)** still shows `InsightService` / `InsightReport` / `ReviewSpendingViewModel`. Those classes exist in the code and are fine to keep, but they belong to the Insights tab, not the Sprint 3 innovative feature. If you want the class diagram to reflect Sprint 3, add: `PreSpendInterventionFlow`, `PreSpendInterventionUseCase`, `WaitReminderScheduler`, `WaitReminderRouter`, `NotificationCenterDelegate`, `InterventionLogRepository`, `InterventionProposal`, `InterventionLog`, `Goal` / `WishlistItem`. Happy to draw this in PlantUML on request.
- Detailed class descriptions (slide 24)** show `BankConnectionService` / `LocalTransactionRepository` / `ReviewSpendingViewModel`. The rubric requires "min 3 classes". Swapping `ReviewSpendingViewModel` out for `PreSpendInterventionFlow` would tie Section 3 directly to the innovative feature without dropping below the minimum.
- Feature matrix (slide 6)** lists "Impulse spend notification & re-decision loop" against the competitors as a Bujet-only . This is exactly the framing for the innovative feature and is already pulling its weight — keep as-is.

## 7. Updated class diagram (slide 21) — PlantUML

The current class diagram still revolves around `InsightService` / `ReviewSpendingViewModel`. The Sprint 3 innovative feature has *its own* class graph that doesn't appear anywhere. The diagram below keeps the Sprint 1 (bank connection) and Sprint 2 (manual transaction) spines from the original — they're correct — and adds the Sprint 3 spine on the bottom half. Stereotypes follow the (V) / (C) / (M) convention you already used in the original slide.

The rubric says **"Class Diagram (Abstract)"**, so I've left out internal helpers (e.g. `BackendAuthClient`, `TransactionImportFlow`) and private fields — only the load-bearing public surface is shown.

```

@startuml ClassDiagram_Bujet
skinparam classAttributeIconSize 0
skinparam linetype ortho
hide empty members

' =====
' Sprint 1 – Bank connection (kept from original)
' =====
package "Sprint 1 – Bank connection" {
    class "ConnectBankView (V)" as ConnectBankView
    class "ConnectBankViewModel (C)" as ConnectBankVM
    class "BankConnectionService (C)" as BankConnService
    class "SecureTokenStore (C)" as SecureTokenStore
    class "BankConnectionRepository (C)" as BankConnRepo
    class "BankConnection (M)" as BankConnection
    class "Account (M)" as Account

    ConnectBankView -right-> ConnectBankVM : observes
    ConnectBankVM -right-> BankConnService : uses
    BankConnService -right-> SecureTokenStore : stores
    BankConnService -down-> BankConnRepo : updates
    BankConnRepo -right-> BankConnection : persists 0..*
    BankConnection -down-> Account : has 1..*
}

' =====
' Sprint 2 – Manual transaction entry (kept from original)
' =====
package "Sprint 2 – Manual entry" {
    class "ManualTransactionSheet (V)" as ManualSheet
    class "ManualTransactionFlow (C)" as ManualFlow
    class "ManualTransactionValidator (C)" as ManualValidator
    class "TransactionRepository (C)" as TxRepo
    class "Transaction (M)" as Transaction

    ManualSheet -right-> ManualFlow : observes
    ManualFlow -right-> ManualValidator : uses
    ManualFlow -down-> TxRepo : savesTo
    TxRepo -right-> Transaction : persists 0..*
}

' =====
' Sprint 3 – Pause & Reflect + Wait re-decision (NEW)
' =====
package "Sprint 3 – Innovative feature" {
    ' Views
    class "PreSpendInterventionSheet (V)" as PRSheet
    class "PauseReflectInputView (V)" as InputView
    class "PauseReflectDecisionView (V)" as DecisionView
    class "WaitConfirmedView (V)" as WaitView
    class "PotCreatedView (V)" as PotView
    class "MainTabView (V)" as MainTabView

    ' Controller / flow
    class "PreSpendInterventionFlow (C)" as Flow

    ' Use case + services
    class "PreSpendInterventionUseCase (C)" as UseCase
    class "SpendingContextProvider (C)" as ContextProvider
    interface "LLMNarrativeService (C)" as LLMService
    class "TemplateNarrativeService (C)" as TemplateLLM
    class "FoundationModelsNarrativeService (C)" as FoundationLLM
    class "WaitReminderScheduler (C)" as Scheduler
    class "WaitReminderRouter (C)" as Router
    class "NotificationCenterDelegate (C)" as NotifDelegate

    ' Repositories
    interface "InterventionLogRepository (C)" as ILogRepoIF

```

```

class "LocalInterventionLogRepository (C)" as ILogRepo
interface "GoalRepository (C)" as GoalRepoIF
class "LocalGoalRepository (C)" as GoalRepo

' Models
class "InterventionProposal (M)" as Proposal
class "InterventionDecision (M)" as Decision
class "InterventionLog (M)"      as Log
class "ImpactSummary (M)"       as Impact
class "SpendingFacts (M)"       as Facts
class "Goal (M)"                as Goal
class "WishlistItem (M)"        as Wishlist

' Sheet → child views
PRSheet *-- InputView
PRSheet *-- DecisionView
PRSheet *-- WaitView
PRSheet *-- PotView

' View ↔ flow
PRSheet -right-> Flow : observes
MainTabView ..> Flow : creates on banner tap

' Flow → use case + services
Flow -down-> UseCase : evaluates via
Flow -down-> Scheduler : schedules
Flow -down-> ILogRepoIF : resolveWait()
Flow -down-> GoalRepoIF : creates pot

' Use case
UseCase -down-> ContextProvider : facts + impact
UseCase -down-> LLMService : narrative
UseCase -down-> ILogRepoIF : appends log

' LLM polymorphism
LLMService <|.. TemplateLLM
LLMService <|.. FoundationLLM

' Notification path
Scheduler ..> NotifDelegate : (UNUserNotificationCenter)
NotifDelegate -right-> Router : sets pendingProposal
Router -right-> MainTabView : observed by

' Repository concretisations
ILogRepoIF <|.. ILogRepo
GoalRepoIF <|.. GoalRepo

' Model relationships
ILogRepo -right-> Log : persists 0..*
GoalRepo -right-> Goal : persists 0..*
Goal -right-> Wishlist : linkedItem 0..1
Log -up-> Proposal : proposalID
UseCase ..> Decision : produces
Decision -up-> Proposal : about
Decision -up-> Impact : numbers
ContextProvider ..> Facts : builds
}

' Cross-sprint reuse (Sprint 3 reads from Sprint 1+2 repos)
ContextProvider ..> TxRepo : reads
ContextProvider ..> ILogRepoIF : reads

@enduml

```

## Notes for the slide

- The diagram is wider than the current one. Suggest splitting across **two slides** — Sprint 1+2 (kept) on one, Sprint 3 on a new slide titled "3.2 Class Diagram — Innovative feature".

- Polymorphism is the genuinely interesting design choice: the `LLMNarrativeService` protocol with two concrete implementations (`TemplateNarrativeService` always available, `FoundationModelsNarrativeService` for iOS 26+ with Apple Intelligence). `LLMServiceFactory.make()` selects at runtime. Worth a one-sentence callout on the slide — this directly evidences "well-argued choice of architectural style" in the rubric.
- Numbers (`ImpactSummary`) never originate from the LLM by construction — they're computed deterministically in `SpendingContextProvider.makeImpactSummary` and passed *into* the prompt. Worth a callout too: it's an explicit design decision to keep the user-facing arithmetic correct even when the model is hallucinating.

## 8. Updated detailed class descriptions (slide 24)

The brief says "*min 3 classes / person*". The current slide has three: `BankConnectionService` (Sprint 1), `LocalTransactionRepository` (Sprint 2), `ReviewSpendingViewModel` (Insights — not part of Sprint 3).

**Recommended swap:** keep `BankConnectionService` and `LocalTransactionRepository` (they anchor Sprints 1 + 2 nicely), and replace `ReviewSpendingViewModel` with `PreSpendInterventionFlow`. That gives you one detailed class per sprint, all three load-bearing.

If you have slide room, drop in `PreSpendInterventionUseCase` and `WaitReminderScheduler` too — they show off the layering (Flow as the MVVM controller, UseCase as the domain orchestrator, Scheduler as the platform-services adapter). All three are written below in the yellow-box format you already used.

### 8.1 PreSpendInterventionFlow (replaces ReviewSpendingViewModel)

```
PreSpendInterventionFlow

useCase: PreSpendInterventionUseCase
goalRepository: GoalRepository
interventionLogRepository: InterventionLogRepository
waitReminderScheduler: WaitReminderScheduler
originalProposalID: UUID?
step: PreSpendInterventionStep
amountText: String
itemDescription: String
category: TransactionCategory
decision: InterventionDecision?
isEvaluating: Bool
isRecording: Bool

submitInput() async -> Void
// Validates the proposal, checks budget exists, then runs
// evaluation via the use case.
start() async -> Void
// Re-decision entry point. Auto-evaluates if the flow was
// opened from a notification tap (originalProposalID set).
chooseBuyNow() async -> Void
// Resolves pending wait as .purchased (re-decision) OR
// appends a fresh log (first decision), then writes a
// Transaction so the user's spend records reflect the buy.
chooseWait(_ duration) async -> Void
// Schedules a UNNotification via the scheduler. On
// re-decision DOES NOT log a new wait (preserves the
// single-record data-integrity NFR).
chooseAddToPot() async -> Void
// Creates a Goal (pot) with a linked WishlistItem;
// resolves pending wait as .skipped on re-decision.
chooseSkip() async -> Void
// Re-decision only. Resolves pending wait as .skipped, no
// pot, no transaction. Powers the "saved by pausing" stat.
```

### 8.2 PreSpendInterventionUseCase (optional — add if room)



```

PreSpendInterventionUseCase

contextProvider: SpendingContextProvider
narrativeService: LLMNarrativeService
interventionLogRepository: InterventionLogRepository

evaluate(_ proposal) async throws -> InterventionDecision
// Builds SpendingFacts from repos, computes the
// deterministic ImpactSummary in Swift, then asks the LLM
// for the prose + suggested action. Numbers never come
// from the LLM by construction.
recordDecision(for decision, action) async throws -> Void
// Appends an InterventionLog audit entry. Separate from
// evaluate() so the presentation layer controls when the
// log is written (e.g. skipped on re-decision to avoid
// double-counting).

```

### 8.3 WaitReminderScheduler (optional — add if room)

```

WaitReminderScheduler

center: UNUserNotificationCenter

requestAuthorizationIfNeeded() async -> Bool
// Lazy permission request; succeeds for already-authorised
// states, returns false on denial.
schedule(for proposal, duration) async -> Void
// Builds a UNNotificationRequest whose userInfo encodes
// the proposal (id, amount, description, category,
// currencyCode) using primitive types only – so the
// NotificationCenterDelegate can reconstruct an
// InterventionProposal on tap without needing the app
// process to have been alive.
triggerInterval(duration, now, calendar) -> TimeInterval
// DEBUG: ~5s for live demo. Release: honours 24h / 7d /
// end-of-month exactly.

```

## 9. Architecture diagram (slide 20) — do you need to change it?

Reading the brief again: §3.1 "Architecture Diagram" is graded under the **Object-Oriented Design** rubric, which asks for "well-argued choice of architectural style for the system". The current diagram shows the three-tier client/server architecture for the **bank connection** path. That's correct and load-bearing for Sprint 1, so **don't replace it**.

But Sprint 3 is *on-device only* — it doesn't touch the backend at all — and that's actually a notable architectural decision worth a callout. Suggest **adding one bullet** to the prose next to the existing diagram, or a small inset diagram, along these lines:

*Sprint 3 (Pause & Reflect) operates entirely on-device. The on-device Foundation Models LLM, local `UNUserNotificationCenter`, and JSON-persisted repositories (`InterventionLogRepository`, `GoalRepository`) keep the user's proposed spend data off the network entirely — no proposal, amount, or category is ever transmitted to the backend or to Apple's servers. This satisfies S3.5 (on-device LLM for privacy) at the architecture level, not just the implementation level.*

PlantUML for an optional Sprint-3 inset (kept small so it can sit beside the existing three-tier diagram):

```

@startuml Architecture_Sprint3_OnDevice
skinparam shadowing false
skinparam componentStyle rectangle

package "iOS device – no network involvement" {

```

```
component "Pause & Reflect UI\n(SwiftUI views)" as UI
component "PreSpendInterventionFlow\n+ UseCase" as Logic
component "Foundation Models\n(on-device LLM)" as LLM
component "UNUserNotification\nCenter" as UN
database "Local JSON store\n(InterventionLog,\n Goal, Transaction)" as Store

UI -down-> Logic
Logic -right-> LLM : prompt(facts, impact)
Logic -left-> Store : append / resolve log
Logic -down-> UN : schedule wait reminder
UN -up-> UI : tap -> deep-link
}

note right of LLM
  Numbers (ImpactSummary)
  are NOT generated by the LLM.
  Computed in Swift, passed
  into the prompt as facts.
end note
@enduml
```

10. Final slide-by-slide change list (consolidated)

| Slide | Section                               | Change                                                                                                                                                                                            |
|-------|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 10    | §2.1 Pause-before-purchase use case   | Redraw with PlantUML in §2.2 above; add <b>Skip</b> use case.                                                                                                                                     |
| 12    | §2.2 Use case table (Pause & review)  | Reword the <b>4iii(alt)</b> alternative to clarify the Skip flow (see §5).                                                                                                                        |
| 17    | §2.3.5 S3.4 description               | Replace placeholder with text in §3.                                                                                                                                                              |
| 18    | §2.3.6 Sprint 3 test cases            | Replace T3.1–T3.8 with T3.1–T3.11 from §4.                                                                                                                                                        |
| 20    | §3.1 Architecture diagram             | Keep current diagram. Add one Sprint-3 on-device bullet (§9), or add the small inset diagram.                                                                                                     |
| 21    | §3.2 Class diagram                    | Either redraw using §7 PlantUML, or split across two slides: keep current Sprint 1+2 diagram, add new Sprint 3 diagram.                                                                           |
| 23    | §3.3 Review Spending sequence diagram | <b>Replace</b> with the Sprint 3 sequence diagram from §2.3.                                                                                                                                      |
| 24    | §3.4 Detailed class descriptions      | Swap <b>ReviewSpendingViewModel</b> for <b>PreSpendInterventionFlow</b> (§8.1). Optionally add <b>PreSpendInterventionUseCase</b> (§8.2) and <b>WaitReminderScheduler</b> (§8.3) if there's room. |

11. Alignment check against the module brief

Brief requirements vs. plan:

| Brief item                    | Required quantity                                | After applying this plan                                                                                     |
|-------------------------------|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| §1.2 Similar systems          | min 3 / person                                   | 3 (Snoop, Emma, YNAB) — unchanged                                                                            |
| §2.1 Use case diagrams        | (no minimum, but should cover the system)        | Top-level + Transaction import + Manual entry + Pause-before-purchase (with Skip) — covers all three sprints |
| §2.2 Use case tables          | (no minimum stated, slide says "min 3 / person") | 3 (Connect bank, Manually record, Pause & review) — unchanged                                                |
| §2.3 Shall-statements + tests | Each sprint must include min 3 NFRs              | S1 NFRs: 3 (S1.3, S1.4, S1.5)<br>S2 NFRs: 1 (S2.3)  — <i>under quota</i><br>S3 NFRs: 3 (S3.4, S3.5, S3.6)    |
| §3.1 Architecture diagram     | 1                                                | 1 (kept) + optional Sprint-3 inset                                                                           |

| Brief item                        | Required quantity | After applying this plan                                                                                                                      |
|-----------------------------------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| \$3.2 Class diagram (abstract)    | 1                 | Updated to include Sprint 3                                |
| \$3.3 Message sequence diagram    | (no count)        | 3 (Connect bank, Manual add, Pause & Reflect re-decision)  |
| \$3.4 Detailed class descriptions | min 3 / person    | 3 if you swap; up to 5 if you add UseCase + Scheduler      |

⚠ **One thing I noticed that's worth fixing while you're in there:** Sprint 2 only has 1 NFR (S2.3: AES-256 secure storage). The brief says "**min of 3 NFRs**" per sprint. Suggest promoting two of the existing S2 statements to NFRs by rewording, e.g.:

- **S2.4** (persistence across restarts) is already non-functional in nature — reclassify as **(NFR)** instead of (FR).
- **S2.5** (works without internet) is non-functional in nature too — reclassify as **(NFR)**.

That gives Sprint 2 three NFRs (S2.3, S2.4, S2.5) without any new content. Tiny edit, removes a rubric risk.